08-1-decisions

accounts-api security assessment · generated from the repository, not hand-edited

> **Section 8 deliverable: biggest decisions, enforcement posture, two blast-radius scenarios worked fully, the detection gap, the regulatory table, and what was cut.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`DECISIONS.md`

Decisions and trade-offs

1. The three I would defend hardest

(a) I spent the budget on identity scoping, not on runtime hardening. The load-bearing controls here are the OIDC `sub` + `job_workflow_ref` pin (`iam/github-oidc-trust-policy.json`) and the rewritten pod policy (`iam/pod-identity-policy.json`). Everything else is a second line. The reasoning is in the threat model's "overrated" paragraph: in this architecture an attacker with RCE already reaches the whole customer dataset through the pod's legitimate identity, so making the identity worth less beats making the container harder to escape. **Trade-off:** these controls are brittle in a way that hardening is not. Renaming the workflow file breaks production deploys with an opaque `not authorized to perform sts:AssumeRoleWithWebIdentity`, and a scoped pod policy means every new AWS call the service needs is a PR against a security-owned file. That friction is real, it lands on other people, and it is the reason policies like this get widened at 2am and never narrowed again. I mitigate it with the break-glass path and by putting the *reason* for each condition in the file — not by loosening the condition.

(b) I chose not to block merges on unfixable vulnerabilities, and not to enforce four admission policies cluster-wide on day one. The SCA gate runs `--ignore-unfixed`: a CRITICAL with no upstream patch does not stop a merge. Four Kyverno policies — restricted security context, image digest, signature verification, secrets-as-env — ship in **Audit** outside `ns/accounts`. SAST warns rather than blocks until 2026-10-15. The reasoning is the same in all three cases and it is about the *survival* of the control, not its correctness. A gate whose only satisfying action is "file a waiver" trains people to waive, and the next real finding gets the same reflex. A policy that breaks four other teams on a Tuesday gets the admission controller uninstalled, and then the correct policy protects nothing. **Compensating controls, because "we decided not to" is not an answer on its own:** the unfixable backlog goes to a non-blocking scan that is trended and reviewed monthly, so it is *visible* rather than *enforced*; SBOM attestations are attached to every digest so the "which running images contain this package" question is answerable in minutes rather than days; every Audit policy is **Enforce in `ns/accounts` today**, which is both a real control on the service that holds the ledger and the evidence I take to the other twelve teams; and each policy carries its promotion criteria in its own annotations with a named date. **Trade-off:** for the length of the rollout, eleven other namespaces can run unsigned, root, tag-referenced containers next to mine. I think that is the right trade against the alternative of having no admission control at all — but it is a trade, and if the cluster is breached through a co-tenant in the next quarter this is the decision that will be examined.

(c) Keyless signing on the Sigstore public good instance. No standing private key exists anywhere — which matters precisely because a keyed cosign key would live in an Actions secret, readable by exactly the T1 attacker the signing is meant to defend against. **Trade-off:** I have made the production deploy path depend on the availability and integrity of a third party's Fulcio and Rekor. If Rekor is down, admission verification fails and — with `failurePolicy: Fail` — deploys stop. If the Sigstore TUF root were compromised, the signature claim is worthless. Both are stated in `docs/02-build-integrity.md` rather than elided, and the real fix (a mirrored TUF root and a private Fulcio/Rekor via `sigstore/scaffolding`) is in §6 below as cut for time.

2. Enforcement posture

gate	day one	promotion trigger
waiver integrity (schema, approver, signature, expiry)	**BLOCK**	already blocking. Has no waiver path of its own, deliberately.
secret scanning, full git history	**BLOCK**, any severity	already blocking. Severity is meaningless for a credential.
IaC — conftest against `policy/rego`	**BLOCK**	already blocking. Small rule set, every rule is data-plane exposure.
policy unit tests (`kyverno test` , `conftest verify`)	**BLOCK**	already blocking.
SCA — fixable CRITICAL/HIGH	**BLOCK**	already blocking.
SCA — unfixable	WARN	promotes only if the distroless migration lands and the standing backlog is under 10.
SAST (Semgrep ERROR)	WARN	2 consecutive weeks of zero findings later triaged as false positives, backlog < 10 each with an owner, median PR-to-green not moved > 20%. Announced date 2026-10-15; if the evidence does not hold, the *ruleset* shrinks and the gate promotes anyway.
Checkov	WARN permanently	never. One tool owns the merge decision; the second exists for coverage. Two blocking IaC scanners means two rulesets disagreeing and an engineer who cannot tell which to satisfy.
Kyverno: host namespaces, hostPath, RBAC wildcards, **privileged RoleBindings**, registries	**ENFORCE** cluster-wide	already enforced; each promoted on a 14-day dry-run whose violation counts are recorded in the policy annotations.
Kyverno: security context, digest, signature, secrets-as-env	**ENFORCE** in `ns/accounts` , AUDIT elsewhere	per-policy criteria in `docs/05-admission-rollout.md` ; promoted namespace by namespace, never cluster-wide in one change.
Kyverno: resource limits	AUDIT	deliberately not queued. Tenancy control, not security; `LimitRange` is the better fix.

What counts as evidence that a gate is ready to promote. Not "it has been in audit for a while". Three things, and the third is the one people skip: (i) a `PolicyReport` or scan report with zero failures across the namespaces in scope for the stated window; (ii) the window contained representative deploy traffic — I count

admission requests, not just failures, because a quiet fortnight over a change freeze is not evidence; (iii) the remaining exceptions are enumerated by name in the policy, each one a line somebody had to write and review, rather than absorbed into a broadened rule. Enumerate reality, then enforce the enumeration.

3. Blast radius

(a) `accounts-api` is compromised via RCE

Reaches: the request-handling process as uid 10001; the mounted DB credential at `/var/run/secrets/db/credential.json` and therefore the full customer table — name, email, phone, IBAN, balance, last-4; the two Kafka topics; the KYC provider; and one Secrets Manager secret. That is the whole prize, and it is reached at step one. This is why I called container escape overrated.

Denied: the node — no `hostPath`, no `hostNetwork`, no privilege escalation, `drop: ALL`, read-only root filesystem, distroless (no shell to land in). Other namespaces' secrets — the pod's Kubernetes RBAC grants one named ConfigMap and nothing else, and the DB credential never transits the Kubernetes API. The rest of AWS — `iam:*` and `sts:AssumeRole` are explicitly denied, so no persistence and no role chaining; `aws:RequestedRegion` denies everything outside `ap-south-1`; and `aws:SourceVpce` makes the stolen credential inert from the attacker's own machine. CloudTrail and GuardDuty cannot be touched. Egress goes to one FQDN through a pinned CA.

Evidence: `accounts_api_credential_read_by_unexpected_process` fires the moment anything other than PID 1 opens the credential file (CRITICAL, pages security on-call, runbook in `runtime/`); `accounts_api_unexpected_process_spawned` on any exec; RDS `pgaudit` shows the queries; CloudTrail shows every API call under the pod role; the image digest in the alert resolves to an SBOM attestation, so we know what was in the image without guessing.

Where the evidence goes dark: *inside* the process. An attacker who runs their payload in-process — a deserialisation gadget, a loaded `.so`, or simply issuing extra SQL through the existing connection pool — spawns nothing, opens no new file, and makes no unusual AWS call. Falco sees nothing. The only signal left is the *shape* of the database traffic, and we do not currently baseline it. That is the same gap as §4.

(b) A maintainer's GitHub account is taken over and a workflow file is modified

Reaches: the source of truth. They can merge (branch protection requires a review, but a second compromised or complicit account, or a self-approving admin, defeats that), so they can put arbitrary code in the image and it will be **validly signed** — the signature attests provenance, not intent, and this is exactly the scenario `docs/02-build-integrity.md` warns against over-reading it for.

Denied: the fast path. A *new* workflow file cannot assume the deploy role, because the trust policy pins `job_workflow_ref` to `.github/workflows/build-release.yml@refs/heads/main` — so `evil.yml` gets an OIDC token that STS refuses. Modifying the existing file still requires a merge to `main` and then passing the `production` GitHub Environment's reviewers before the `:environment:production` claim is minted at all. `permissions: {}` plus SHA-pinned actions removes the compromised-dependency variant. A fork or PR gets no token that matches any condition.

Evidence: the commit and its signature; the GitHub audit log (login IP, device, whether MFA was satisfied); the Environment approval record naming who approved; `AssumeRoleWithWebIdentity` in CloudTrail carrying the run id; the Fulcio certificate in the Rekor transparency log recording the workflow ref and the time — which is

genuinely useful here, because it is an append-only record the attacker cannot edit even with full repo admin.

Where it goes dark: GitHub's audit log is GitHub's. An attacker with org-owner rights can disable branch protection, delete the audit log stream, and rewrite history — and our retention of that log is a GitHub setting, not something we hold independently. We stream it to S3, but the stream is configured in the same account we assume is compromised. Also dark: if the takeover results in a *subtly* malicious change rather than an obvious one, nothing in this design distinguishes it from a normal release. Provenance is intact and useless. The control that actually addresses (b) is two-person review with hardware-backed commit signing, which is organisational and is in §6.

4. The attack path I would currently not detect

Bulk exfiltration through legitimate channels: a compromised pod (or a valid credential) reading the customer table at scale and sending it out through the permitted KYC egress path, or an attacker paging the authenticated API through the ALB with a stolen customer token. Both use allowed identities, allowed destinations, allowed ports, and valid TLS. Every control in this repository approves of them. Falco sees no new process and no new socket; the NetworkPolicy and the egress gateway see the destination they were configured to permit; the audit trail records a series of successful, authorised calls.

Cost to close, roughly in the order I would buy it:

1. **RDS query-shape alerting** — `pgaudit` to CloudWatch with a rule on unbounded `SELECT` and on row counts beyond the 99th percentile for that user. ~2 engineer-days plus modest CloudWatch spend. Catches the database half.
2. **Per-pod egress byte-rate baseline** — Istio telemetry, alert on bytes-out per pod deviating from the KYC request profile. ~3 days. Catches the network half in the bulk case; misses a slow drip.
3. **Field-level authorisation in the application** — the KYC code path can read only the four fields it needs, enforced in the data-access layer rather than by convention. Weeks, and it is application work, not platform work. This is the only one that is *prevention* rather than detection, and it is the one I would actually argue for. Detection of exfiltration through an allowed channel is a consolation prize; not having the data reachable from that code path is the answer.
4. **DLP on the egress gateway payload.** Technically possible (Envoy ext_authz / WASM inspecting the request body). I would not buy it: it breaks TLS end-to-end, adds a component that sees plaintext PII on the hot path, and is trivially defeated by compression or encoding. It looks like a control and is mostly a new attack surface.

5. Regulatory framing

control built	obligation it serves
`provider "aws" { region = "ap-south-1" }`, single-region KMS keys, `DenyEverythingOutsideApSouth1` in the pod policy, logs pinned to `ap-south-1`	**Data residency** — RBI *Storage of Payment System Data* requires payment data to be stored in India. Enforced in the identity layer, not only in the deployment config, so it survives a misconfiguration.
CloudTrail org trail with `enable_log_file_validation`, Object Lock (COMPLIANCE, 7y) in a separate log-archive account, SCP denying `s3:DeleteObject`	**Audit-trail integrity** — the digest chain makes the log tamper-evident, and the separate account with Object Lock means the principal who acted cannot delete the

control built	obligation it serves
	record of it. This is the difference between logging and evidence.
35-day RDS PITR, `deletion_protection`, ECR `NobodyDeletesImages`, 400-day log retention, 7-year archive	**Retention** — RBI/Companies Act record retention, plus the ability to reconstruct what ran on a given date, which is what an examiner actually asks for.
Signed, expiring waivers with a two-person approval matrix (`waivers/APPROVERS.yaml`), and the break-glass 72-hour follow-up	**Access review / change control** — every exception to a control has a named approver, a cryptographic signature, an expiry, and a ticket. This is the artefact an auditor samples.
Redaction at serialisation, no card data in logs, last-4 treated as in-scope, `log_statement = 'ddl'`	**Cardholder-data scope** — PCI DSS 3.3/3.4/10.x. Keeping card data out of logs is also what keeps the logging estate *out of PCI scope*, which is worth more than the control itself.
Falco CRITICAL → PagerDuty with a named on-call and a three-step runbook; 5-minute emergency credential rotation	**Breach-notification readiness** — DPDP Act's 72-hour notification and RBI's 6-hour incident report both start at *detection*. Without a named owner and a rehearsed first 15 minutes, the clock runs while people work out who is responsible.

6. What I cut, and Monday

Cut for time, in the order I regret them:

- **Field-level authorisation in the application** (§4.3). The single highest-value remaining control and the only one that would prevent, rather than detect, the exfiltration path I cannot see.
- **A private Sigstore (mirrored TUF root, self-hosted Fulcio/Rekor).** Removes a third-party availability and trust dependency from the production admission path.
- **Hardware-backed commit signing with enforced two-person review**, which is the actual control for blast-radius scenario (b) and is organisational rather than technical — which is why it is slow, not why it is unimportant.
- **A CI-side `conftest` pass over rendered Kubernetes manifests.** Today the Kyverno policies only run at admission plus in unit tests; the same rules applied to rendered manifests in CI would turn a cluster rejection into a red pipeline. Half a day of work and I should have done it.
- **A Semgrep rule banning PII in URL path and query parameters** (§6 of the redaction doc). Currently a review convention, and a review convention is not a control.
- **Rehearsal of the rotation fallback.** Documented, not tested. Untested rotation is not rotation.
- **DNS-layer egress control** (CoreDNS policy plugin), leaving DNS tunnelling open.
- **Deliberately deprioritised, not merely cut:** gVisor/Kata sandboxing, a WAF ruleset, and cluster-wide `LimitRange` capacity work. Reasoning in the threat model — the first two defend against the second step of chains whose first step is already addressed, and the third is not a security control.

Monday morning, in a real rollout: the CI-side `conftest` pass over rendered manifests, before anything else. It is half a day, it makes the entire section 5 policy set apply *before* deploy rather than at admission, it turns an opaque webhook rejection into a legible pull-request comment, and it removes the argument the other

twelve teams will otherwise make — that admission control is a thing that breaks their deploys with no warning. Getting them to accept the promotions in §2 is the whole rollout, and that half-day is what makes the ask reasonable.